G10 · MARINE DEBRIS CLASSIFICATION · DEEP LEARNING PROJECT

Project journey: the problems we hit, how we fixed them, and what each fix led to

This is our team's record of the project, from reading the assignment to the final training run. It covers how we built one model that classifies marine debris from four camera viewpoints, what broke along the way (data, architecture, code and fine-tuning), which early wins turned out not to be wins, and why the final models look the way they do.

Updated 8 Nov 2026 with the final (v3) results | Deadline 9 Nov 2026

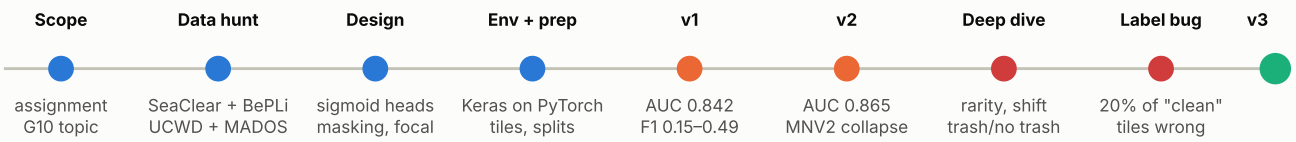
Summary

- **Goal:** one network that looks at a sea photo from any of four viewpoints (underwater, shore, drone, satellite), says which viewpoint it is, and lists the debris in it. A basic CNN is the baseline; MobileNetV2 (ImageNet) is the transfer-learning model.
- **Viewpoint was solved almost immediately** (99.6% accuracy). Telling debris types apart was the hard part: macro F1 stayed between 0.15 (underwater) and 0.55 (drone) even when ranking quality (AUC) was decent (0.82 to 0.95).
- **Most of the progress came from data, not architecture:** removing park/concrete images from the drone set, cutting shore photos into smaller tiles (shore AUC 0.75 → 0.81), and finally discovering that our tiling rule had labelled **1 in 5 "clean" underwater tiles that actually showed trash**.
- **The final run (v3)** retrained both models on corrected labels, with a dedicated "trash present?" neuron per head and a fixed MobileNetV2 fine-tuning stage (learning rate 1e-5 instead of the 1e-4 that collapsed it).
- **Outcome:** on the same test photos, underwater "is there trash?" went from **62–71% to 82–88%** (ranking). Drone trash detection reached **96% accuracy**. The small CNN (1.42M parameters vs MobileNetV2's 6.18M) **ties MobileNetV2 on shore and drone and beats it on satellite**; MobileNetV2 only clearly wins underwater. The remaining limits come from the data, not the build (section 14).

1 · Timeline at a glance
2 · Understanding the assignment
3 · Finding four datasets
4 · Designing the architecture
5 · Setting up the environment
6 · Preparing the data
7 · v1: first training, first illusion
8 · v2: fixes from the v1 logs
9 · MobileNetV2: promise, then collapse
10 · Why the scores stayed low
11 · Trash or no trash?
12 · The labelling bug
13 · v3: the final run
14 · v3 results: what the final run proved
15 · Final comparison: every metric
16 · Celebrations that turned into losses
17 · Where the code failed us
18 · What each indicator told us
19 · Lessons and future work
20 · Try it: the web app
Appendix · Numbers

1 Timeline at a glance

Each problem we solved exposed the next one. The arrows are the actual order things happened in.



What led to what

Accuracy 99.6% but F1 0.15 → learned accuracy hides rare classes → tuned thresholds → saw val/test mismatch
Shore underfit → BePLi 2×2 → 3×3 tiles → shore AUC +0.06 → per-tile F1 fell (harder tiles) → judged at photo level
MobileNetV2 stage 2 collapsed → lr 1e-4 too high + lever pinned stale head → lr 1e-5 + lever reset at stage 2
Binary "any trash?" weak underwater → audited tiles vs annotations → 30% rule mislabels big objects → v3 relabel + trash neuron

Blue: setup phases. Orange: training runs. Red: diagnosis. Green: the final run.

2 Understanding the assignment

With the extended deadline we had one week. Our group's (G10) topic is **marine debris classification**. The assignment asks for:

Requirement	How we covered it
A transfer-learning model (MobileNetV2 or ResNet50)	MobileNetV2 pretrained on ImageNet, two-stage training (frozen, then fine-tuned)
A baseline (AlexNet or a basic CNN)	Our own 4-block CNN (1.42M parameters), trained from scratch
Accuracy, precision, recall, F1, confusion matrix, ROC-AUC	All computed per class on the test set, plus photo-level and trash-vs-clean views
Dataset (or link) with justification	Four public datasets, one per viewpoint (section 3)
Complete, organized, commented code	Data script + one notebook per experiment, all code inline in the notebooks
Presentation incl. literature (2025–26), results, future work	Built from this report; results and lessons are in sections 7–19

We also set up a GitHub repository early, with a README, a requirements file and a .gitignore.

3 Finding four datasets

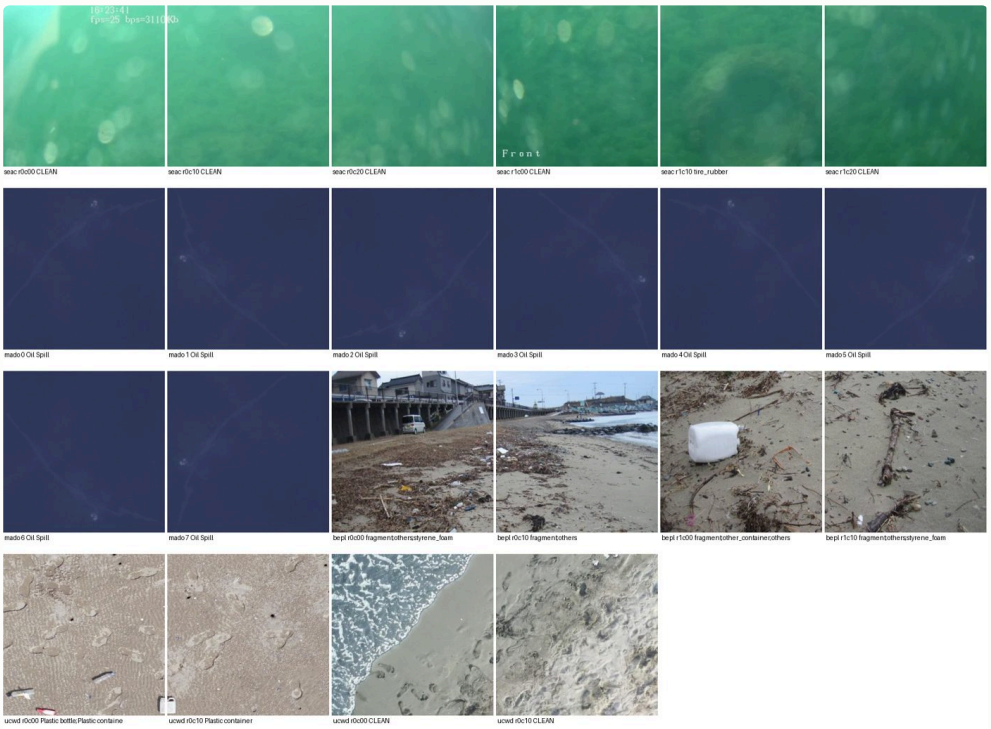
PROBLEM

The dataset we started with, **SeaClear**, is entirely underwater. A marine-debris classifier that only works underwater misses most of where debris is actually monitored: beaches, drone surveys, satellites.

WHAT WE TRIED

- **Seanoe** collections looked excellent, but the satellite options were weak.
- A **NASA** dataset: it wouldn't download, and when we looked closer it had a bigger problem for us. Its annotations didn't fit image-level classification, and the marine-debris class was tiny.
- **Drone:** first the Dal Lake FloPWD set (a lake, not the sea), which we replaced with **UCWD**. It has 7 litter classes, and sigmoid outputs handle images with several labels.
- **Satellite: MADOS** (Sentinel-2, pixel masks). Classes like shallow water and non-annotated pixels are useless as outputs, so they became background.
- **Shore: BePLi v2**, beach plastic with 13 classes.

Viewpoint	Dataset	Images	Labels	Our classes
Underwater	SeaClear	8,610	COCO boxes + outlines, 40 raw classes	16 (after merges)
Shore	BePLi v2	3,722	COCO, 13 classes	13
Drone	UCWD (coastal sources only)	3,169 kept of 4,071	COCO, 7 classes	7
Satellite	MADOS	2,803 patches	pixel masks, 15 classes	7 (after merges)

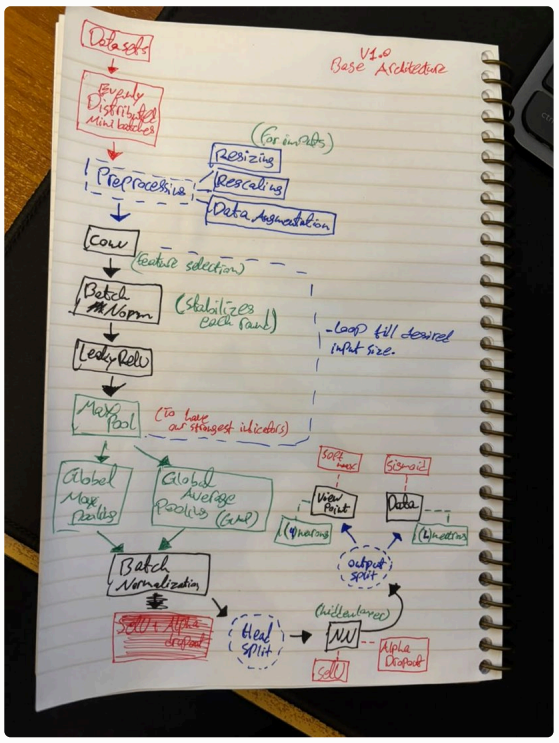


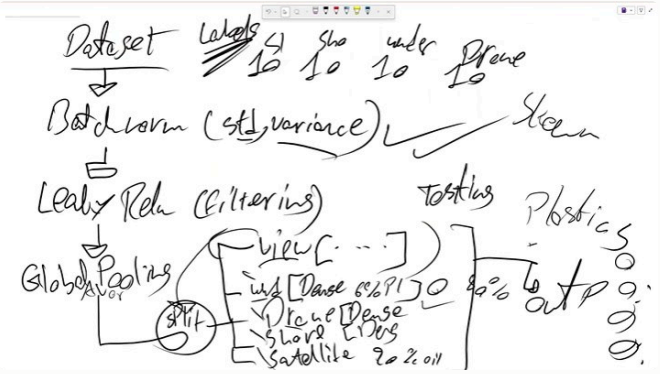
Sample prepared tiles from each dataset. Underwater (green water), satellite (dark blue Sentinel-2 patches), shore and drone. Four very different looks for one network.

4 Designing the architecture

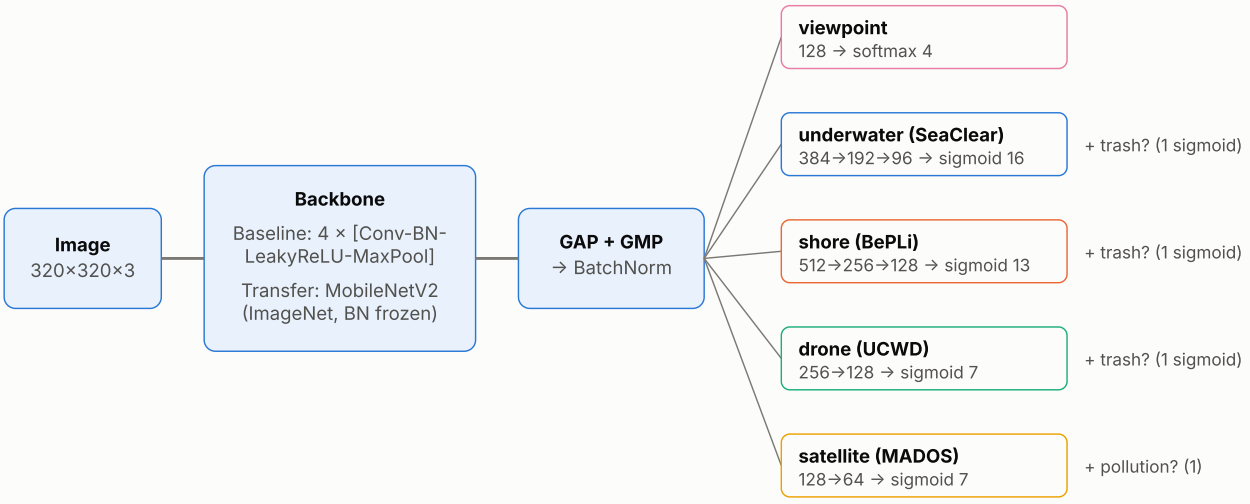
We designed the model on paper before writing any code, and discussed each piece as a group. Every decision answers a concrete problem:

Question / problem	Decision	Why
Softmax forces exactly one class per image, but a tile can hold a bottle *and* a rope	**Sigmoid** outputs, one per class (multi-label)	Each class is its own yes/no; softmax would make them compete
Four datasets with different class lists	**One shared backbone + one head per dataset** + a viewpoint head	One model hosts all four datasets: shared visual features, separate class lists
An underwater image has no shore labels: is "no bottle" a real no?	**Loss masking**: each image trains only the viewpoint head and its own dataset's head	Avoids teaching the shore head that every underwater image is clean
Most tiles are clean; classes are rare	**Binary focal loss** ($\gamma = 2$), balanced batches, clean tiles re-sampled to 20%	Plain cross-entropy learns "always no"
Tiny debris (MADOS median 4 debris pixels)	**Global average + global max pooling**	Average pooling washes out a few bright pixels; max keeps them
Activations and normalization	LeakyReLU + BatchNorm in the backbone; SELU + AlphaDropout in the heads; inputs scaled to −1…+1	Stable training; self-normalizing dense heads
Uneven dataset sizes	**Adam / AdamW**; each batch draws a fixed number of tiles from each dataset (square-root balanced)	No dataset dominates; small ones aren't drowned
Noisy early gradients	Batch-size warm-up: **32 for 3 epochs, then 96** (Smith et al. 2018)	Small, noisy batches early; stable ones later
MADOS has only 1,433 training patches	8 saved orientations (4 rotations × flip), training only	Satellite images have no "up"
What if a big dataset swamps the rest?	Capping every dataset at MADOS size, kept only as a fallback	Never needed





Where it started. Left: the hand-drawn "V1.0 Base Architecture": evenly distributed mini-batches → preprocessing (resizing, rescaling, augmentation) → a Conv–BatchNorm–LeakyReLU–MaxPool block looped to the input size → global max + global average pooling → BatchNorm → SELU + AlphaDropout → the head split into a softmax viewpoint output and sigmoid dataset outputs. Right: the whiteboard version of the split into viewpoint, underwater, drone, shore and satellite heads. The final model (below) is still this design.



Each image's loss uses only the viewpoint head and its own dataset's head (masking).
Head sizes and the "trash?" neurons are the v3 versions: 47 class outputs + 4 trash neurons = 51 outputs.

The final (v3) architecture. Both models share everything after the backbone, so the baseline-vs-transfer comparison changes only one thing.

We wrote all the numbers into a spec workbook (`G10_Marine_Debris_Data_Spec.xlsx`): inputs, tile counts, the training pool, the batch mix, MADOS class counts, class lists, augmentation, model settings, and a change log. A counting script over all 2,803 MADOS masks produced the satellite class table.

5 Setting up the environment

PROBLEM

The plan said TensorFlow, but TensorFlow has no native GPU support on Windows anymore, and we didn't want WSL. The GPUs are two RTX 5060 Ti (Blackwell, compute capability sm_120), newer than stable PyTorch builds supported.

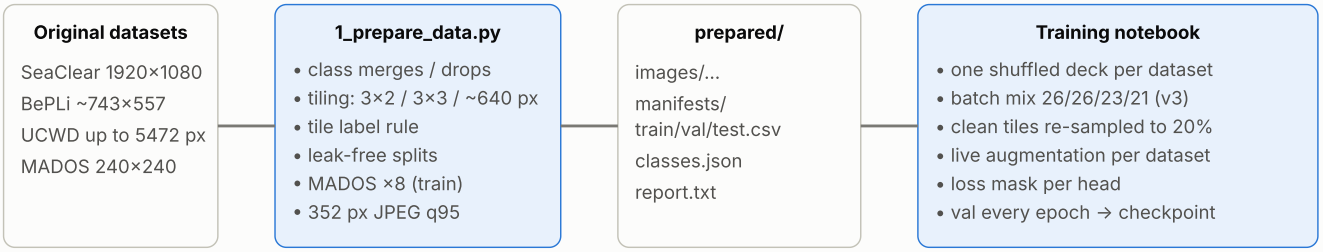
FIX

Keras 3 on the PyTorch backend, natively on Windows, with a PyTorch nightly build for CUDA 12.8 that includes sm_120. We wrote it as plain notebooks; an environment-check notebook confirms both GPUs, real GPU math and Keras's backend.

THEN

- **Speed test:** about 464 images/s at 224 px. A channels-first experiment crashed because the input shape was still channels-last, and once fixed it made no difference.
- **Code layout:** shared code first lived in a `g10.py` module. We dropped it so that every notebook carries all of its code inline and can be read and discussed on its own.
- **Machine:** Ryzen 7 9700X (16 threads). We use **7 loader threads per notebook**, which leaves the computer usable while two notebooks train.

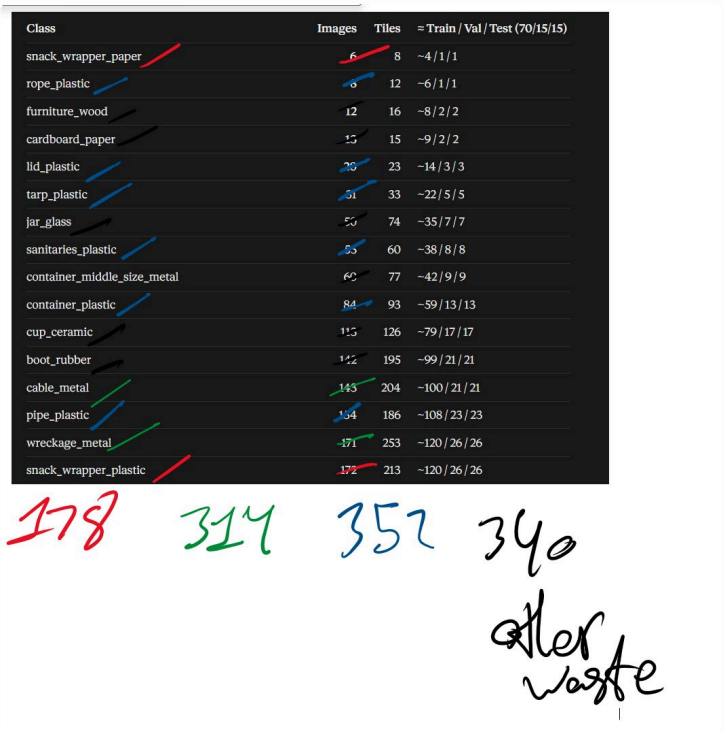
6 Preparing the data



Data flow. Validation and test always use original tiles only, never augmented copies.

6.1 Class merges

Several SeaClear classes had only a few dozen examples. We merged them by meaning (rope_fiber + rope_plastic → rope; branch_wood + furniture_wood; cable + wreckage → metal_debris; lids, tarps, sanitaries, containers, pipes → other_plastic; cardboard, jars, metal containers, ceramic cups, boots → other_waste) and set animals, plants, ROV parts and "unknown" aside as not trash. MADOS: algae types merged, foam + waves & wakes merged, sea snot into natural organic material, jellyfish dropped (6 training patches).



The hand-marked merge plan for the smallest SeaClear classes (colours = target groups).

6.2 Problems found while extracting

PIXELATION	Small BePLi photos came out as 213×160 tiles that got upscaled and looked blocky.

FIX

No upscaling when saving; small tiles kept at native size; JPEG quality 95 without colour subsampling; Lanczos when shrinking; one single resize at load time. Model input raised from 224 to **320** so small objects appear 1.43× larger. That was a deliberate choice over smaller tiles, at about 2× the compute.



An early tile with visible upscaling artefacts. It led to the quality settings above.

POISONED DRONE DATA

While checking tiles we found that UCWD contains grass, parks and concrete floors, plus phone close-ups that aren't drone shots at all.

FIX

Dropped 902 images by source (BATCH: parks/grass/concrete; GOPR/PHOTO/CAMERA: handheld ground shots; CD 3024×3024: close-ups) and re-split the rest. Verified afterwards: 0 such tiles left in any split.



UCWD samples by source. Some are clearly not coastal (grass, kerbs, concrete), which would teach the drone head the wrong backgrounds.

LEAKAGE

Video frames: SeaClear and UCWD contain consecutive frames from the same videos. A random split puts near-identical frames in train and test, so the scores look great and mean nothing. UCWD's official split leaked exactly this way.

FIX

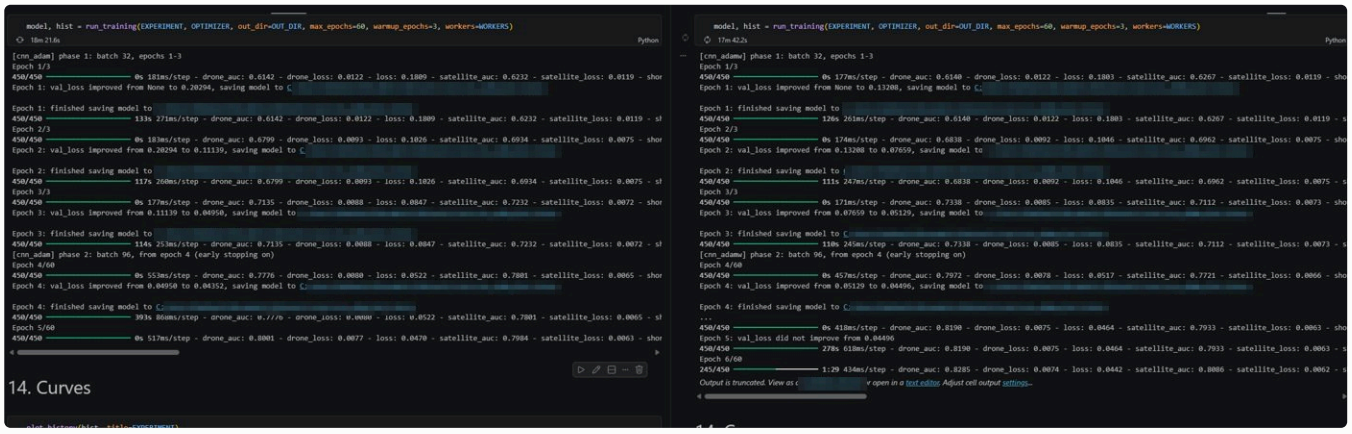
SeaClear: **video-block split** (blocks of 50 consecutive frames, 2 frames dropped at each boundary, 300 random assignments tried, the most class-balanced kept). UCWD: re-split with each video kept whole.

6.3 Code and tool problems in this phase

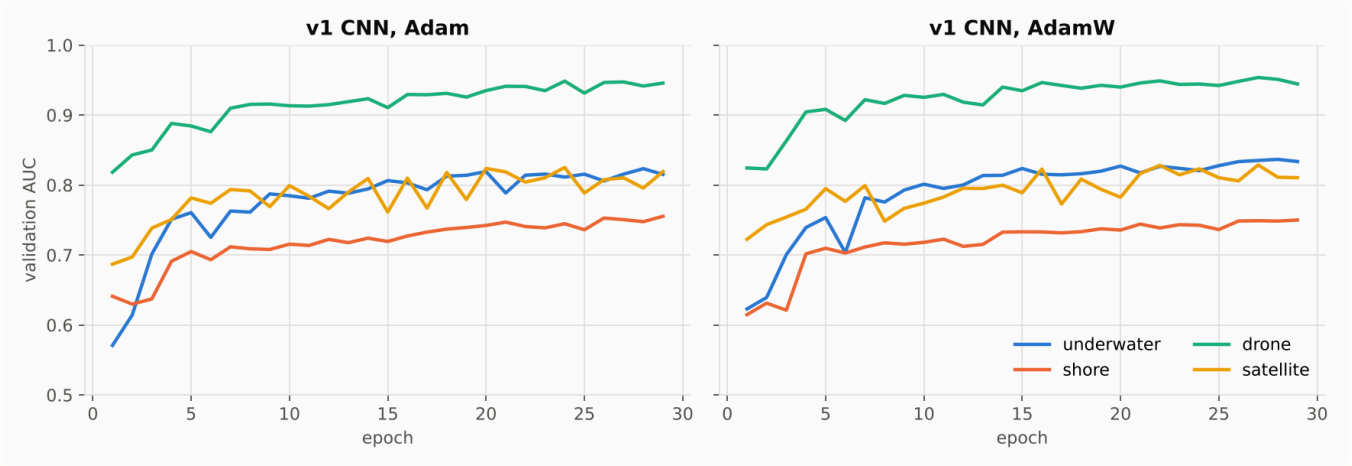
Symptom	Cause	Fix
Re-preparing one dataset wiped the others from the manifests	Manifest written from the current run's rows only	Merge in the existing rows of datasets not being processed
The "overwrite tiles" option was ignored	Windows starts worker processes fresh; a global variable isn't inherited	Pass it through an environment variable
Data loader crashed on start	An extra argument passed into Keras's `PyDataset`	Made it an explicit parameter
Spreadsheet updates kept disappearing	Excel had the file open and saved its old copy over the new one	Close Excel with "Don't Save" before updates

7 v1: first training, first illusion

Two notebooks ran in parallel, one per GPU: the CNN with **Adam** and with **AdamW**. They used 2×2 BePLi tiles, heads 256→128→64, and monitored validation loss.



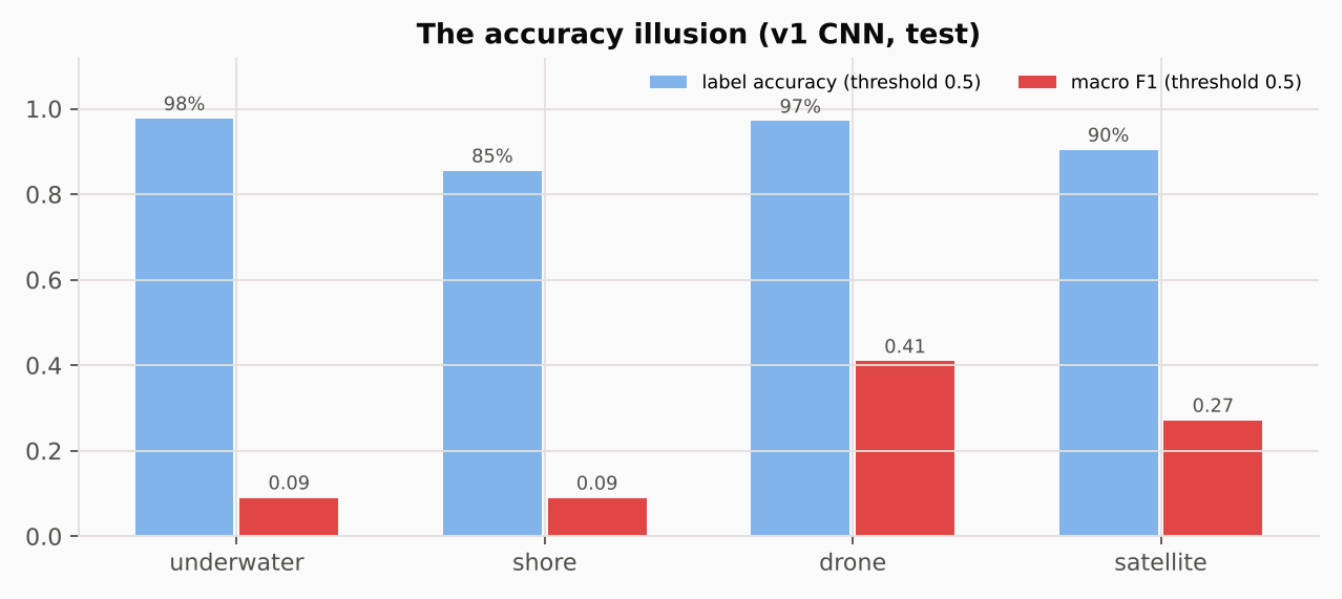
The v1 training log. Losses were falling and viewpoint accuracy was near 99%, and at the time we thought we had hit the target.



v1 validation AUC per head. Drone ~0.94, underwater and satellite ~0.82, shore stuck at ~0.75. Adam and AdamW are practically identical.

THE ILLUSION

On the test set, viewpoint accuracy was 99.6% and per-label accuracy was 85–98%, yet **macro F1 was 0.15 underwater** and under 0.5 everywhere.



Why accuracy looked great while F1 stayed under 0.5. When a class appears in 1.4% of tiles, a model that always answers "no" is 98.6% accurate with F1 = 0. Accuracy rewards the "no"s; F1 only counts the "yes"es.

LED TO

- **Thresholds tuned per class on validation**, test touched once. That raised F1 (underwater 0.09 → 0.15, shore 0.09 → 0.38) and exposed the val/test class-mix mismatch (section 10).
- **Shore** was the outlier: train AUC 0.77 ≈ validation 0.75, so it was underfitting, not overfitting. Causes: tiny objects, cluttered tiles, vague classes ("fragment", "others").
- We agreed to finish the runs and study the logs before changing the data again.

8 v2: fixes from the v1 logs

v1 evidence	v2 change
Shore underfit; BePLi objects ~12 px at the input	**BePLi 3×3 tiles** (objects ~18 px; labels per tile 2.3 → 1.4)
Total loss dominated by the easy viewpoint head	Monitor `val_trash_auc` = mean validation AUC of the 4 trash heads
Satellite overfit (train 0.95 vs val 0.81); longest gap between its improvements was 7 epochs	**Satellite lever**: freeze its head after 8 epochs without improvement
Different dataset sizes, shore underfit, underwater mild overfit	Heads sized per dataset: shore 512-256-128, underwater 256-128-64 + dropout 0.2, drone 256-128, satellite 128-64

Before training we spotted another problem: some BePLi 3×3 tiles show only sea walls and concrete revetments, which helps explain why shore was the weakest head.



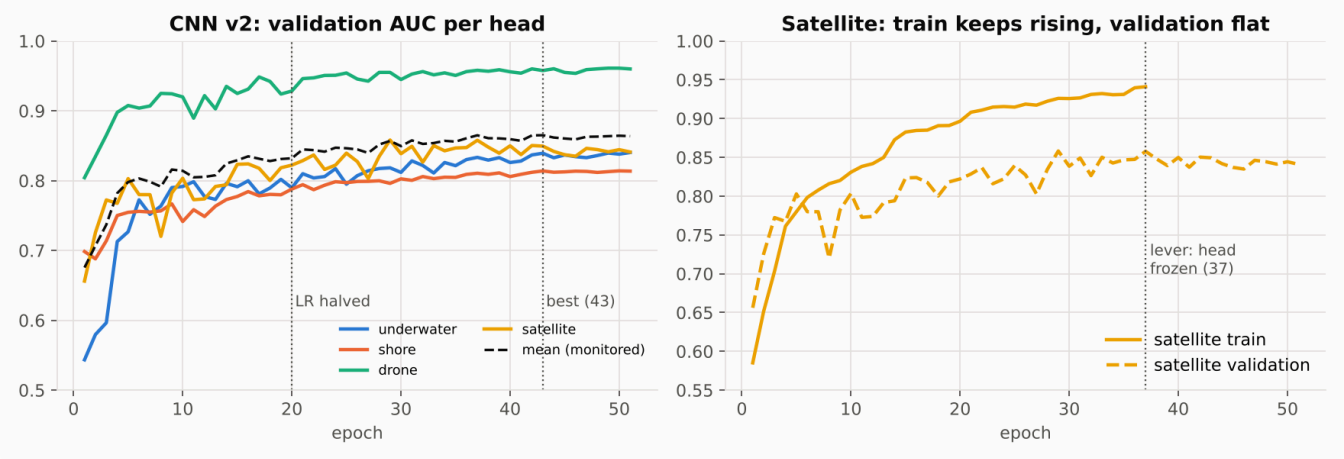
BePLi tiles from the sea-wall photos. After discussing it we kept them: they are real shore backgrounds and teach the shore head what *isn't* debris.

CODE FAILURE

The first satellite lever switched the head's layers to `trainable=False` mid-training. The optimizer had been built for a fixed set of weights and crashed (*"optimizer_variables must be 1:1"*).

FIX

Freeze by **masking its loss to 0** and **putting its best weights back after every batch**. Same effect, and the optimizer never sees a change.

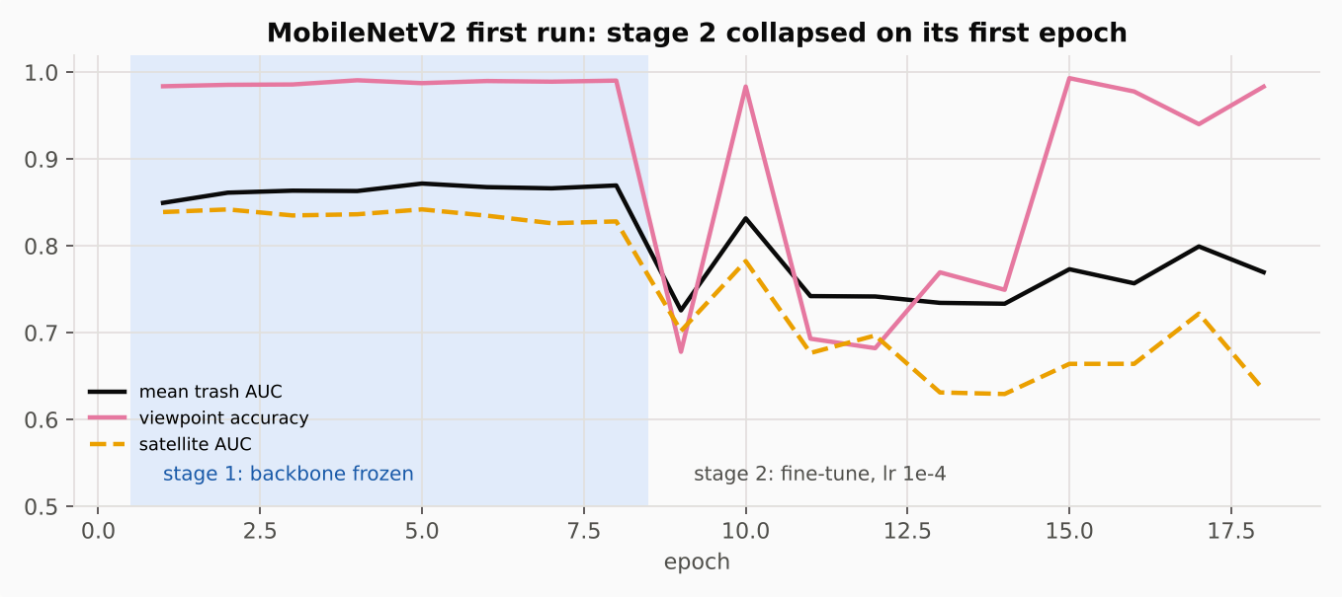


CNN v2. Left: every head improved, and a learning-rate halving around epoch 20 gave a visible jump; best epoch 43 (mean 0.865). Right: the satellite lever fired at epoch 37. Training AUC stops (frozen), validation stays in the same band: it stopped further overfitting, but didn't raise the score.

v2 test results (CNN): AUC improved on every head (underwater 0.823, shore 0.808, drone 0.941, satellite 0.849). Shore per-tile F1 *dropped* (0.38 → 0.33) even though its AUC rose by 0.06. The 3×3 tiles form a harder, different test set. At photo level shore improved (0.545 → 0.573), so we judged shore there.

9 MobileNetV2: promise, then collapse

MobileNetV2 has the same heads, data and losses. Stage 1 freezes the ImageNet backbone and trains only the heads; stage 2 unfreezes from `block_13_expand` (BatchNorm kept frozen) at a lower learning rate.

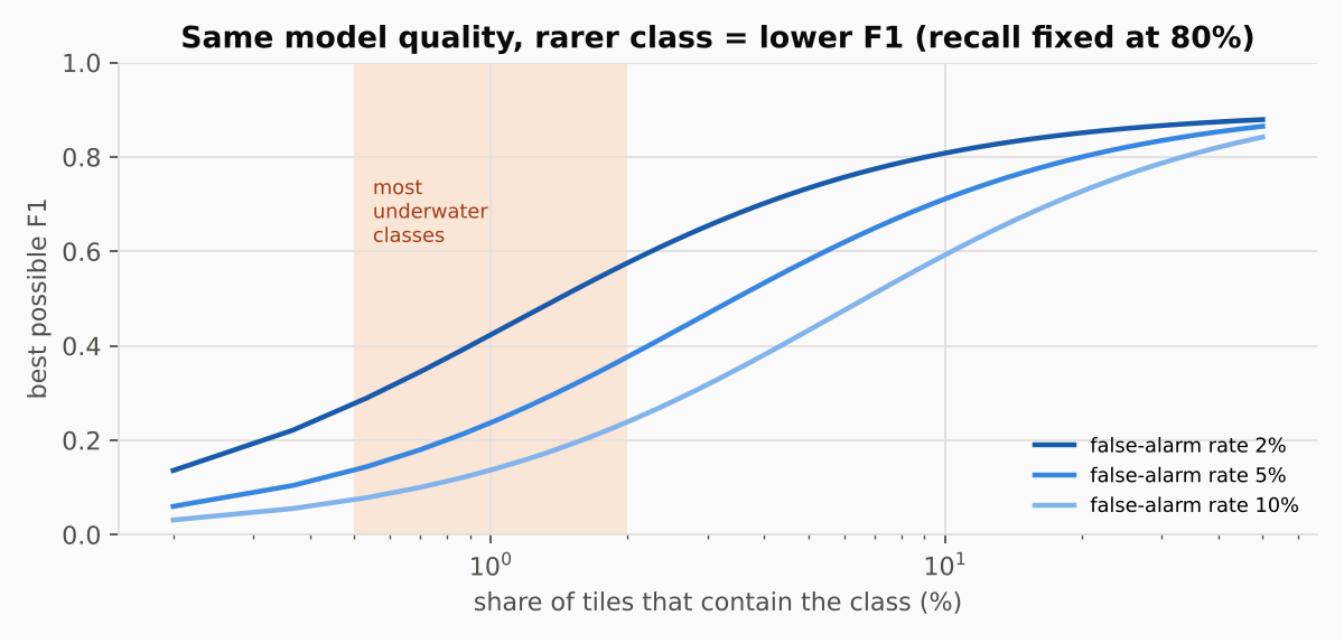


Stage 1 reached mean trash AUC 0.872 within 5 epochs (CNN v1 needed 20+ epochs for 0.842). On the first fine-tuning epoch, viewpoint accuracy fell from 0.99 to 0.68 and every head dropped.

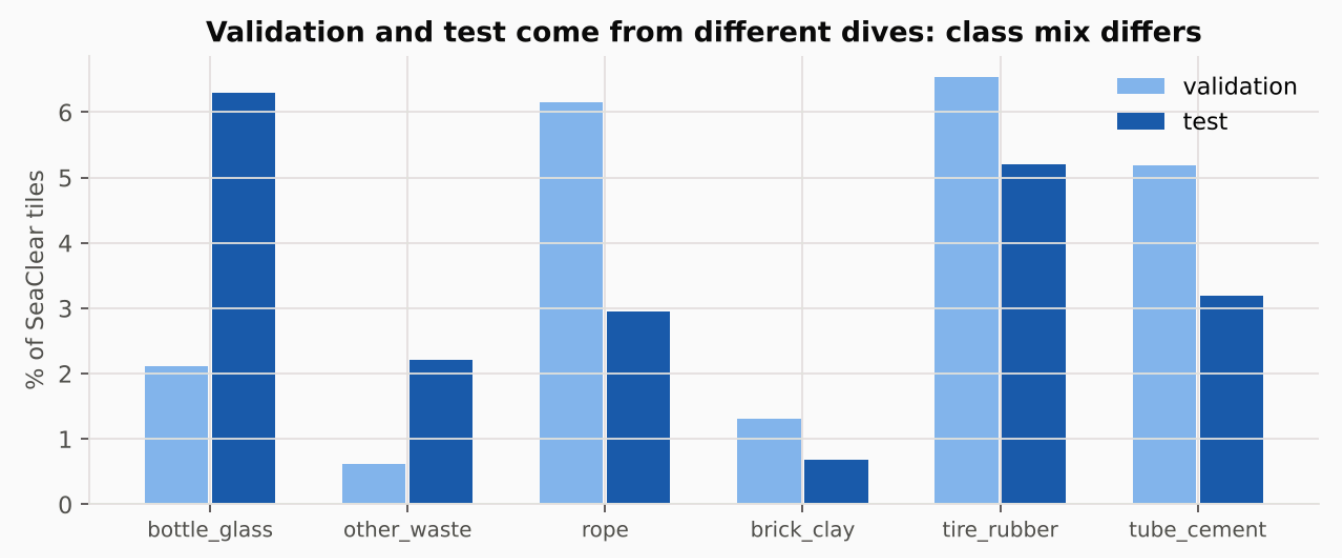
CAUSE 1	Fine-tuning learning rate **1e-4** was too high. Large updates destroyed the pretrained features (Keras's own guide uses 1e-5).
CAUSE 2	The satellite lever, carried over from stage 1, kept restoring **stage-1 head weights onto a backbone that had changed**. Satellite AUC sank to ~0.63.
FIX (V3)	Stage 2 at **1e-5**, and a **fresh lever** that starts counting at stage 2. The saved best model from the first run is stage 1 (epoch 5), and it's what we evaluated.

10 Why the scores stayed low

After v2 the scores were still disappointing, and we asked ourselves whether we had done something wrong or simply didn't have enough data. Three effects explain most of the low F1.

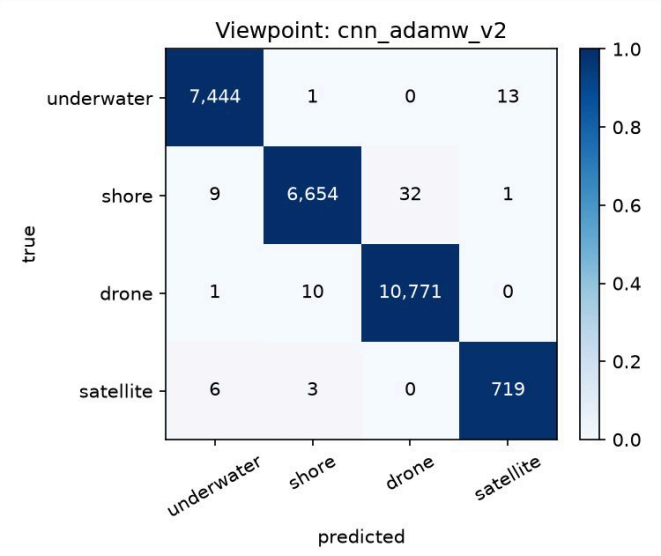


Rarity. With the same recall and false-alarm rate, F1 collapses as a class gets rarer. With 75 real positives among 7,458 tiles, a 5% false-alarm rate still means about 370 false alarms. metal_debris had AUC 0.93 but F1 0.07.



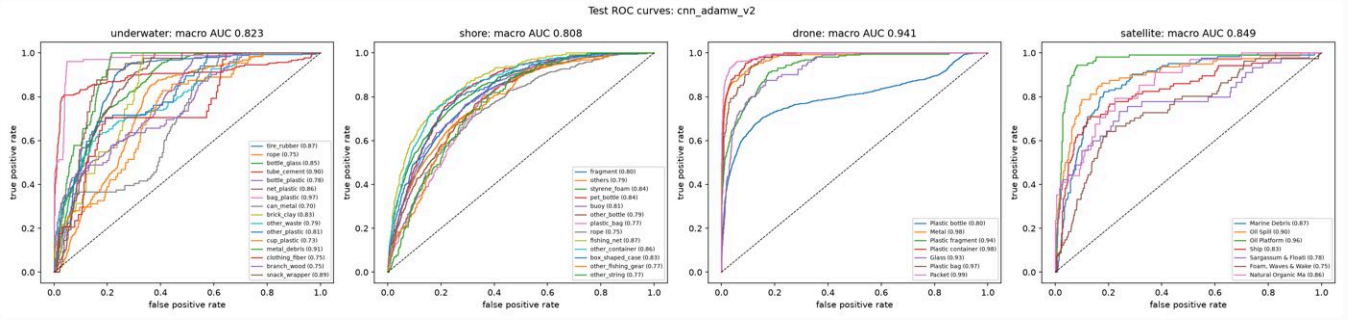
Thresholds don't transfer. Leak-free splits mean validation and test come from different dives, with different class mixes. brick_clay's validation-tuned threshold (0.12) flagged a third of the test tiles: recall 100%, precision 2%.

Our assessment. The architecture work wasn't wasted: the multi-task framework, masking, sigmoid outputs and focal/balancing are the structure both models share. The fine-tuning details (SELU vs ReLU, exact head sizes, Adam vs AdamW) moved results by a point or two at most. The big factors were the data (too few *independent* examples per class: many frames of the same few videos), the conversion from detection boxes to tile labels, and the scope (43 fine-grained classes across 4 domains). The leak-free splits make our numbers lower but honest.





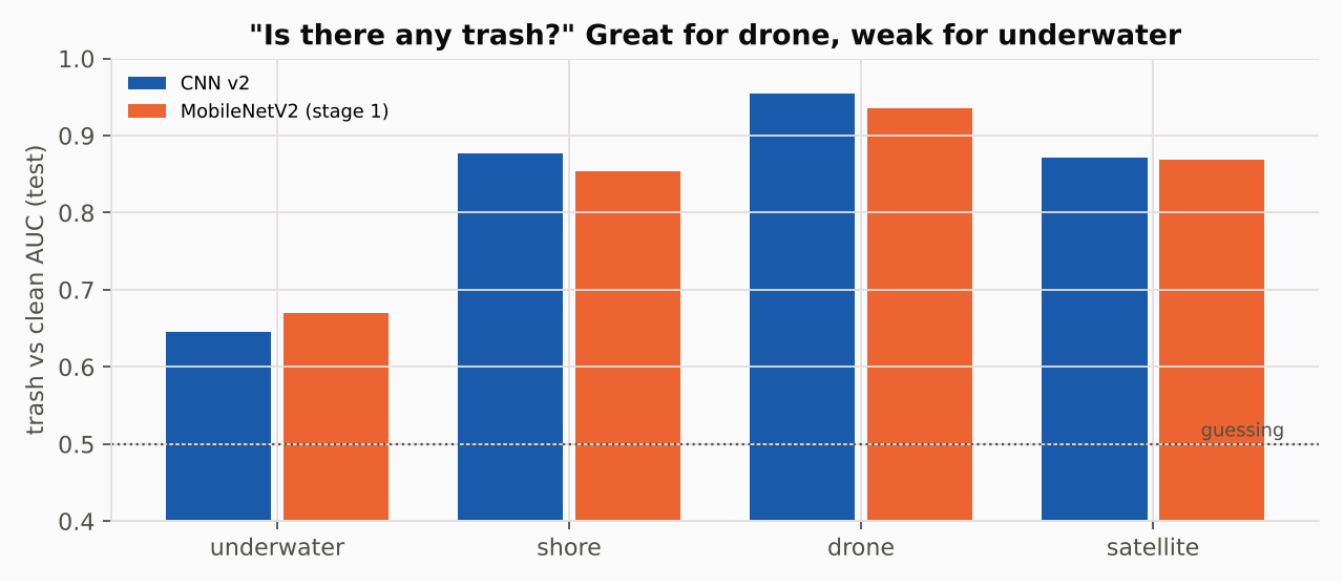
CNN v2 on test. Left: viewpoint is essentially solved. Right: per-class confusion for underwater; rare classes drown in false positives.



CNN v2 test ROC curves per head. Ranking is decent everywhere and strong for drone.

11 Trash or no trash?

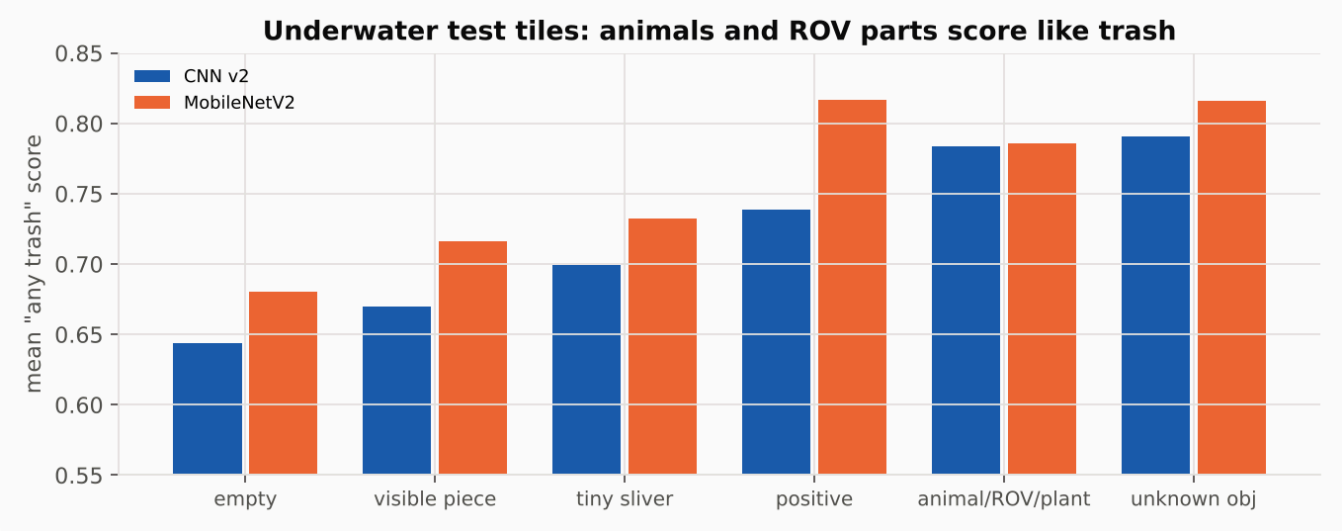
One idea in our discussion was to reduce the problem to "trash or no trash", expecting the numbers to jump. We tested it without retraining: each head's class scores were combined into one "any trash" score.



Trash-vs-clean AUC on test. Drone is excellent (0.95, 91% accuracy), shore and satellite are decent. Underwater is barely above guessing (CNN 0.65, MobileNetV2 0.67; noisy-OR combination 0.75).

TRAP

Shore binary F1 0.84 looked great, but 64% of shore tiles contain trash, so answering "always trash" already scores 0.78. We added an "F1 if always yes" column to every binary table. Photo-level binary was meaningless too, because 89–100% of photos contain trash.

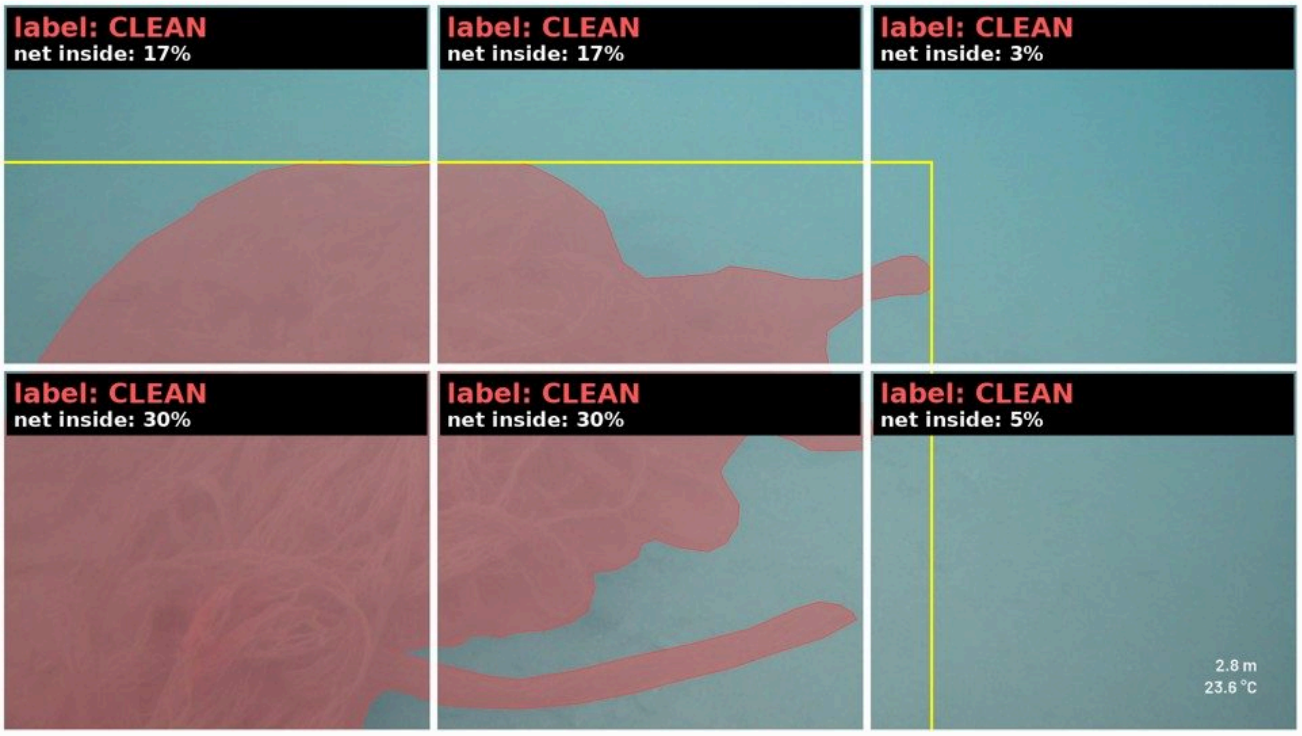


Underwater test tiles grouped by what is *really* in them (from the original annotations). Both models score fish, urchins and ROV parts as high as real trash: they learned "object = trash". "Empty" tiles also score high.

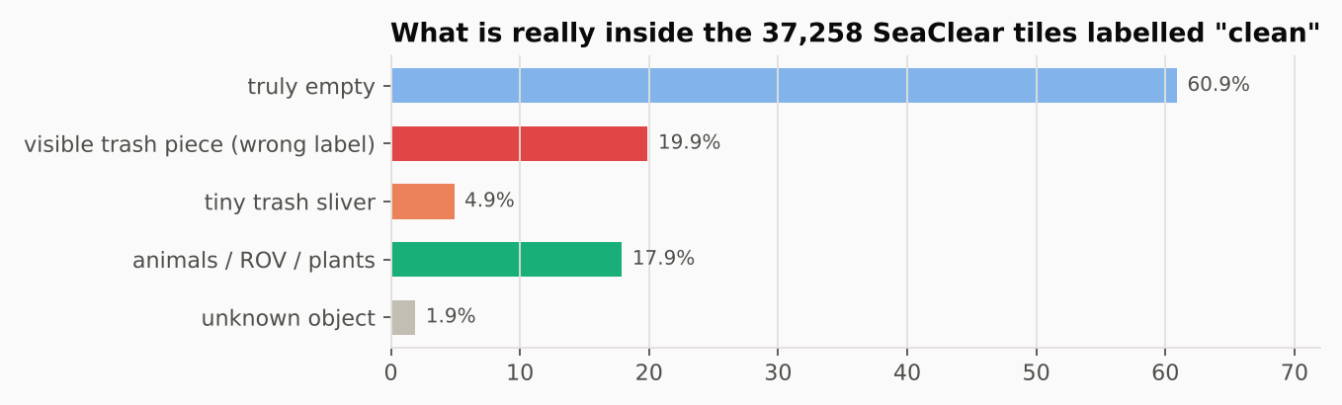
LED TO

When two different models fail at the same thing, suspect the labels. We audited every SeaClear tile against the original annotations.

12 The labelling bug



A real test photo: a fishing net (red outline) cut into our 6 tiles. Our rule said a tile gets a label only if ≥30% of the object's box is inside. The net's slices are 17%, 17%, 3%, 29.9%, 29.9% and 5%, so **every tile was labelled CLEAN**.

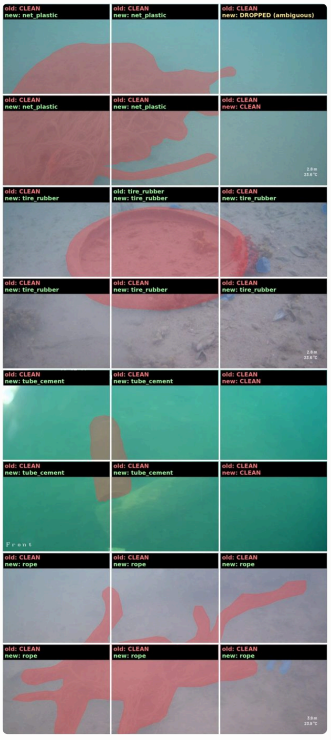


Audit of all 37,258 SeaClear tiles labelled clean: **19.9% showed a clearly visible piece of trash**. The rule failed on big objects (nets, ropes, tires, cement tubes) spanning several tiles. BePLi had the issue on only 4.7% of clean tiles (its objects are small).

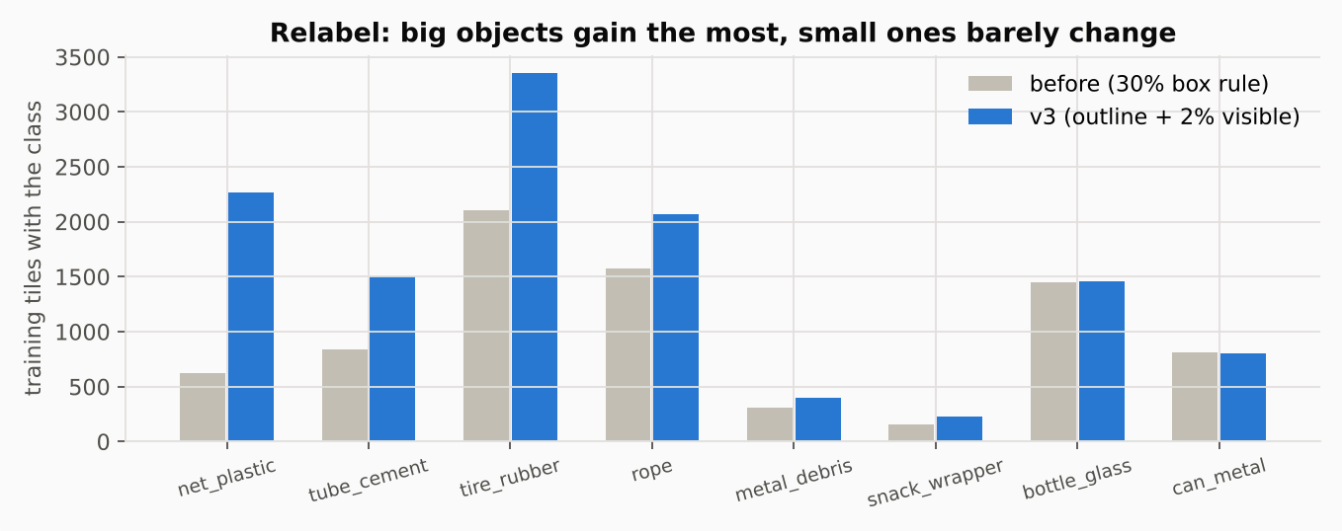
The same rule was used for train, validation and test. So the wrong labels taught the model the wrong thing, and graded it wrongly when it was right.

FIX (V3)

- Measure on the object's **outline**, not its box.
- An object counts if **≥30% of it is inside, or its visible part covers ≥2% of the tile**.
- Tiles with only a sliver (0.1–2%) or an unknown object are **dropped** (3,219 tiles).
- The same rule applies to every split. Images and split are unchanged: the old and new code produce an identical split.



Old vs new label for each tile on four photos (net, tire, cement tube, rope). Several "CLEAN" tiles now carry the right class; one ambiguous tile is dropped.



Training tiles per class before and after. Big objects gain the most (net_plastic ×3.7, tube_cement ×1.8, tire_rubber ×1.6); small objects barely change. SeaClear clean share: 74% → 61%.

13 v3: the final run

Change	Reason
SeaClear relabelled (section 12)	Correct labels for training and grading
Underwater head 384→192→96 (rule: 512 × √(labels ÷ shore's labels)); drone kept at 256→128	Underwater gained 40% more labels; drone's data didn't change and it fitted well. Changing it would confound the comparison
Batch mix 26/26/23/21 (√ of the new pools)	Same balancing rule applied to the new pool sizes
One **"trash present?" neuron** per head (sigmoid, BCE, loss weight 0.5, reported, not monitored)	Freedom to use either the fine-grained or the binary answer. One neuron is enough: P(no trash) = 1 − P(trash)
Satellite trash = Marine Debris or Oil Spill	Algae is natural, not debris (NOAA: debris is manufactured or processed material), and is the main look-alike for plastic in satellite images. Ships and platforms are structures, not pollution
MobileNetV2 stage 2 at lr 1e-5, fresh lever at stage 2	Fixes the v2 collapse
Names: `cnn_adamw_v3` (third CNN run), `mnv2_adamw_v2` (second MobileNetV2 run)	Version per model

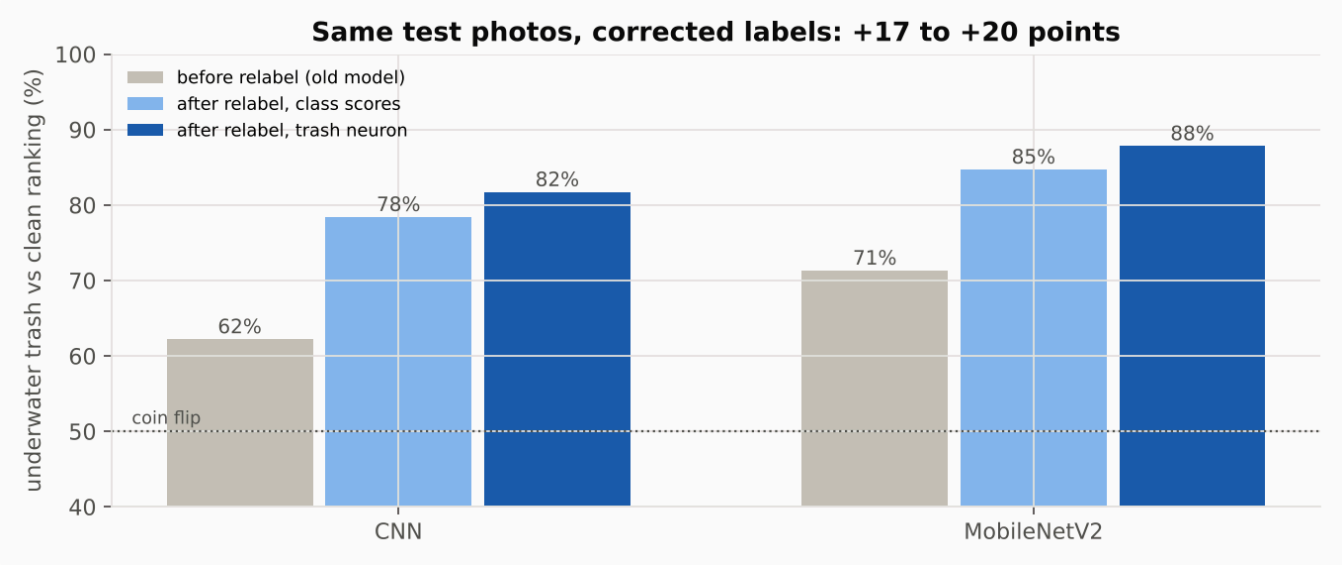
Before launch, every script was tested end to end on small data: the relabel (full run and SeaClear-only), both notebooks (including the lever firing and the stage-2 reset), the trash neuron targets and masks, and the evaluation notebook (including detecting that cached predictions belong to old label files). The real relabel report matched the prediction exactly: 3,219 dropped tiles, clean share 60.9 / 60.0 / 62.1%.

Expectation, stated in advance (the outcome is in section 14): underwater AUC +0.02 to +0.05; not a transformation. Removing the ambiguous test tiles alone moved the trash/clean AUC by only +0.015, so the real gain has to come from training on correct labels. Both models also confuse animals with trash, which relabelling doesn't fix.

14 v3 results: what the final run proved

All numbers below are on the **test set with the corrected labels**. To measure the relabel fairly, the old models were re-graded on the same corrected test photos using their saved predictions.

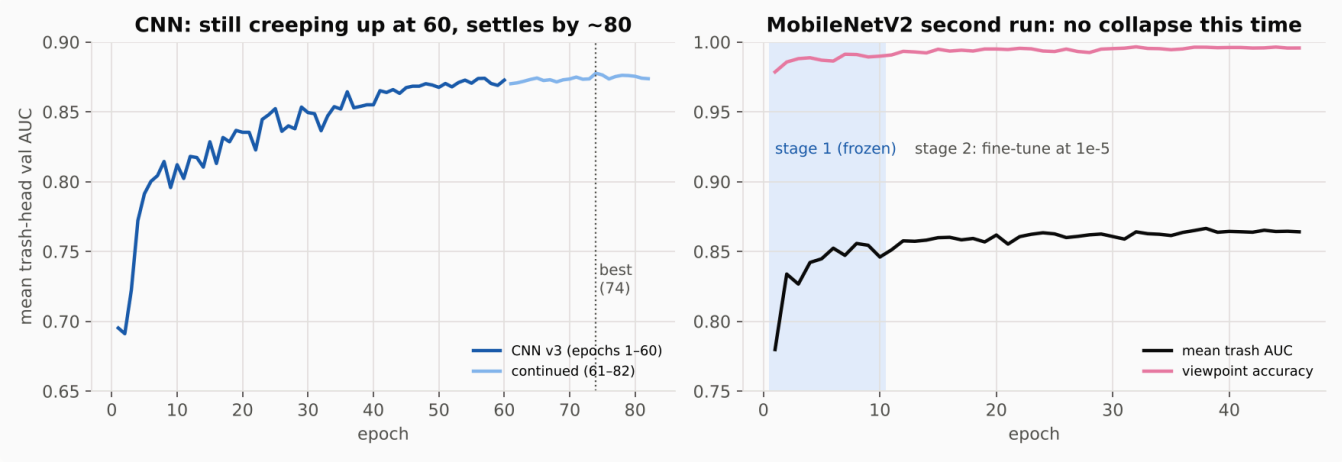
14.1 The relabel was the biggest single improvement of the project



Underwater "is there trash in this tile?", ranking score on the same test photos. Before the fix the CNN was close to a coin flip (62%). After retraining on corrected labels: 78% from the class scores and **82% from the trash neuron**; MobileNetV2 went from 71% to **88%**. The forecast had been +2 to +5 points; the real gain was +17 to +20, because the bug sat exactly on the trash-vs-clean boundary.

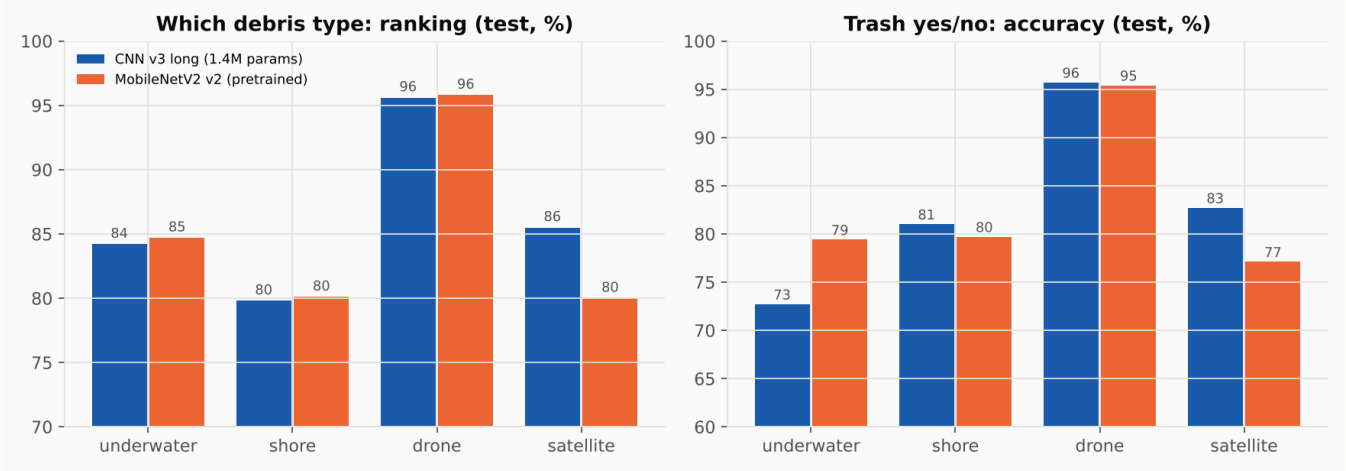
The fine-grained underwater classes improved only modestly (CNN debris-type ranking 82% → 84%), as forecast: the fix mostly corrected which tiles have trash, not which kind.

14.2 Both training fixes worked



Left: the CNN reached its 60-epoch limit still improving, so we continued it from its best checkpoint (same data, learning rate and frozen satellite head) as `cnn_adamw_v3_long`. It peaked at epoch 74 (validation 87.4% → 87.8%) and early-stopped at 82: this is the CNN's ceiling. Right: MobileNetV2's fine-tuning stage at 1e-5 ran smoothly with viewpoint above 99% throughout. The v2 collapse is fixed, though fine-tuning itself added only about +1 point.

14.3 Final comparison: baseline vs transfer learning



Test set, final models. Left: ranking quality for the debris type. Right: accuracy of the trash yes/no neuron.

Test (%)	CNN v3 long	MobileNetV2 v2	Verdict
Underwater: type ranking / F1	84.2 / 21.8	84.7 / 27.1	**MobileNetV2, clearly** (net F1 71 vs 47, tire 67 vs 53, plastic bag 51 vs 50)
Underwater: trash yes/no accuracy / ranking	72.7 / 81.7	79.4 / 87.8	
Shore: type ranking / F1 · trash accuracy	79.8 / 31.5 · 81.0	80.1 / 31.9 · 79.7	Tie
Drone: type ranking / F1 · trash accuracy	95.6 / 58.0 · 95.7	95.8 / 60.0 · 95.4	Tie
Satellite: type ranking / F1 · trash accuracy	85.5 / 54.9 · 82.7	80.0 / 45.2 · 77.1	**CNN, clearly** (Oil Spill F1 77 vs 60)
Viewpoint accuracy	99.5	99.6	Tie
Parameters	1.42M	6.18M	CNN 4× smaller

What the results prove.

- **The ceiling is the data, not the build.** Fixing labels moved underwater trash detection by 17–20 points. Training 22 more epochs moved things by about 1 point. Swapping the whole backbone for a pretrained one changed little outside underwater. Rare debris types (cups, wrappers, other plastics) stay under ~15% F1 for *both* models: too few independent examples.
- **A small task-built CNN is competitive.** With the same heads, losses and data, the 1.42M-parameter CNN ties the pretrained MobileNetV2 on shore and drone and beats it on satellite.
- **Transfer learning pays off where the pretraining images resemble the target:** underwater photos are ordinary camera images (murky ones, where strong pretrained features help most). Satellite patches (240 px Sentinel-2) look nothing like ImageNet, and there the model trained from scratch wins.
- **The trash neuron earned its place:** drone 99% ranking and 96% accuracy; underwater 82–88% ranking.

Hands-on test. In our own tests through the web app (section 20), the predictions on uploaded photos looked accurate. That's a qualitative check, not a measured score: photos from other places and cameras than the four datasets would need a proper labelled test to quantify.

15 Final comparison: every metric

The assignment's metrics (accuracy, precision, recall, F1, confusion matrix, ROC-AUC) for the two final models on the **test set with the corrected labels**. Thresholds were tuned on validation; the test set was used once. All values are percentages.

Reading the metrics. *Accuracy*: share of correct yes/no answers. *Precision*: of the tiles the model flagged, how many were right. *Recall*: of the real cases, how many it found. *F1*: the balance of precision and recall. *ROC-AUC*: how well the scores rank positives above negatives, independent of any threshold (50 = guessing). *Exact match*: tiles where every class was answered correctly.

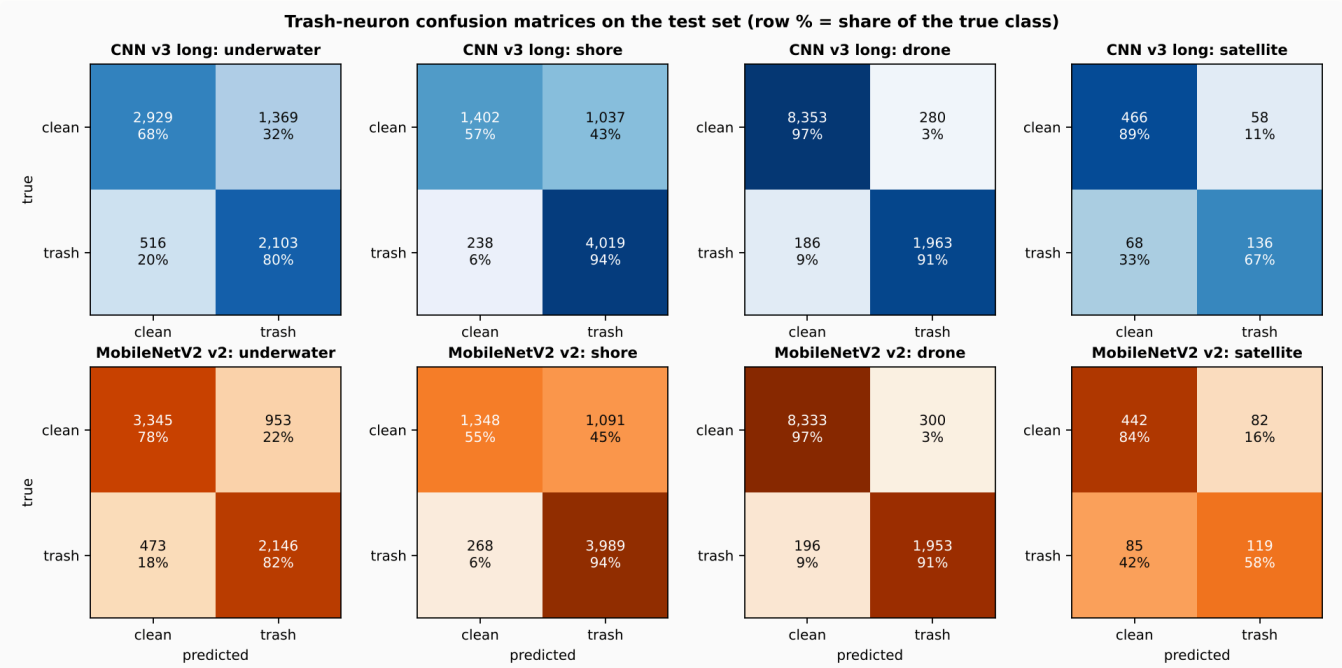
15.1 Which kind of debris (per class, averaged over the head's classes)

Head	Model	Accuracy	Precision	Recall	F1	ROC-AUC	Exact match	Winner
underwater	CNN v3 long	89.7	18.5	42.2	21.8	84.2	38.0	MobileNetV2
	MobileNetV2 v2	93.7	29.7	41.3	27.1	84.7	57.7	
shore	CNN v3 long	82.4	24.2	52.9	31.5	79.8	35.3	Tie
	MobileNetV2 v2	84.0	24.9	49.0	31.9	80.1	33.2	
drone	CNN v3 long	97.7	61.3	56.3	58.0	95.6	87.7	Tie
	MobileNetV2 v2	97.9	67.9	55.1	60.0	95.8	88.2	
satellite	CNN v3 long	89.0	53.8	62.0	54.9	85.5	45.1	CNN
	MobileNetV2 v2	86.9	46.2	50.2	45.2	80.0	37.8	
viewpoint	both	CNN 99.5 · MobileNetV2 99.6 (4-way accuracy)						Tie

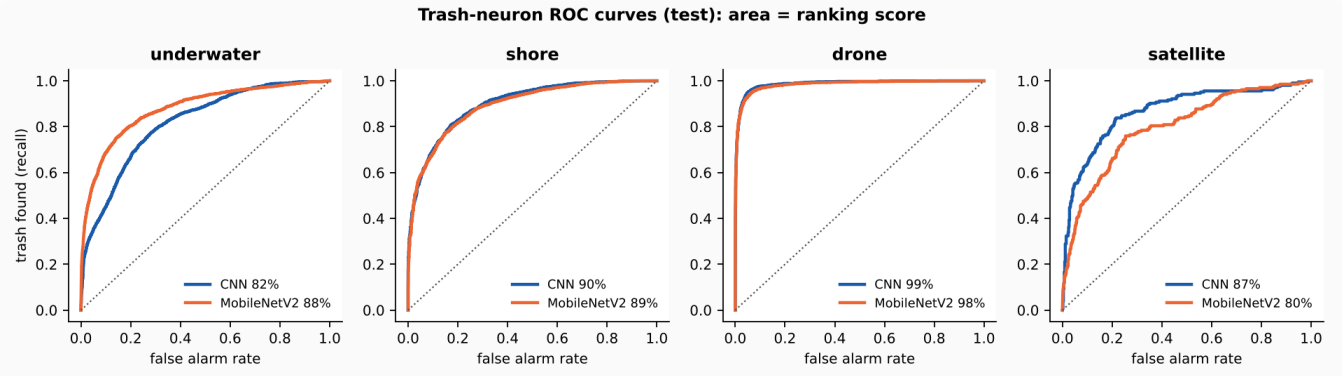
Accuracy is high for every head (82–98%) while macro F1 is much lower: most labels are "no", so accuracy rewards the many easy negatives (section 7). F1 and ROC-AUC are the columns that matter. Precision is the weak spot on underwater and shore: rare classes collect false alarms.

15.2 Is there trash? (the dedicated trash neuron)

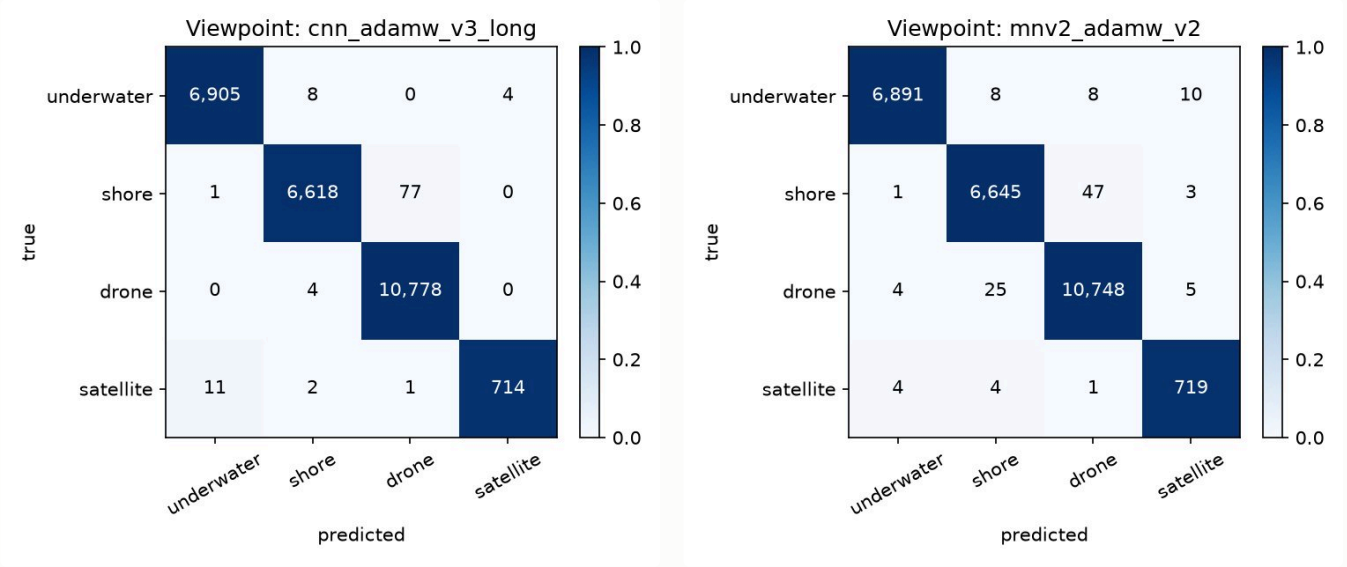
Head	Model	Accuracy	Precision	Recall	F1	ROC-AUC	TP	FP	FN	TN	Winner
underwater	CNN v3 long	72.7	60.6	80.3	69.1	81.7	2,103	1,369	516	2,929	MobileNetV2
	MobileNetV2 v2	79.4	69.2	81.9	75.1	87.8	2,146	953	473	3,345	
shore	CNN v3 long	81.0	79.5	94.4	86.3	90.0	4,019	1,037	238	1,402	Tie
	MobileNetV2 v2	79.7	78.5	93.7	85.4	89.3	3,989	1,091	268	1,348	
drone	CNN v3 long	95.7	87.5	91.3	89.4	98.7	1,963	280	186	8,353	Tie
	MobileNetV2 v2	95.4	86.7	90.9	88.7	98.4	1,953	300	196	8,333	
satellite	CNN v3 long	82.7	70.1	66.7	68.3	86.8	136	58	68	466	CNN
	MobileNetV2 v2	77.1	59.2	58.3	58.8	79.8	119	82	85	442	



Confusion matrices for the trash neuron. Top row CNN, bottom row MobileNetV2. Each cell shows the tile count and the share of its true class. Drone is near-perfect for both; underwater is where MobileNetV2's lower false-alarm count (953 vs 1,369) makes the difference.



ROC curves for the trash neuron per head. The curves overlap on shore and drone; MobileNetV2 sits higher underwater and the CNN sits higher on satellite.



Viewpoint confusion matrices on the test set. Left: CNN v3 long. Right: MobileNetV2 v2. Both are nearly diagonal (99.5% and 99.6%).

Scoreboard. Debris type: MobileNetV2 wins underwater, the CNN wins satellite, shore and drone are within 2 points. Trash neuron: MobileNetV2 wins underwater (F1 75 vs 69), the CNN wins satellite (F1 68 vs 59), shore and drone tie. Viewpoint: tie. With 4× fewer parameters, the CNN wins or ties 3 of 4 heads.

16 Celebrations that turned into losses

Celebration	What it really was	What we did
"We hit the target" (v1 log)	Viewpoint and label accuracy were high because most labels are "no"	Switched to per-class F1, AUC and tuned thresholds
99.6% viewpoint accuracy	Real, but the easy part: four datasets look completely different	Stopped monitoring total loss (viewpoint-dominated); monitor trash AUC
MobileNetV2 stage 1 at 0.872 in 5 epochs	Fine-tuning then collapsed (lr 1e-4)	lr 1e-5, lever reset
v2 looking promising at epoch 29	It was; v2 went on to 0.865. But test F1 stayed low on rare classes	Diagnosed rarity and threshold shift
Shore AUC +0.06 with 3×3 tiles	Per-tile F1 fell, because the test tiles changed	Compared at photo level
The satellite freeze "stopped the overfitting"	Half true: it stopped the train/val gap growing, but validation was already flat and the score didn't rise	Reported as neutral on score, positive on stability
Trash vs clean "would jump to match MobileNetV2"	Jumped for drone; underwater stayed near guessing; shore's F1 was mostly the base rate	Audit → found the labelling bug
(one that held) The relabel, forecast at +2 to +5 points	Delivered +17 to +20 points on underwater trash detection	Kept, and checked it on the same test photos before claiming it

17 Where the code failed us

Where	Failure	Root cause	Fix
Speed test	Crash	channels_first with a channels-last input	Proper test notebook; layout made no difference
prepare_data	Other datasets vanished from manifests	A single-dataset run wrote only its own rows	Merge existing rows
prepare_data	"Overwrite tiles" option ignored	Windows spawn doesn't inherit globals	Environment variable
Data loader	Crash on start	Unknown kwarg passed to `PyDataset`	Explicit parameter
Satellite lever	Optimizer crash	Toggling `trainable` mid-fit	Loss mask + weight pinning
MobileNetV2 stage 2	Collapse	lr 1e-4; lever pinned stale weights	lr 1e-5; fresh lever
Tile labelling	20% of "clean" underwater tiles wrong	30%-of-box rule fails on big objects	Outline + 2%-visible rule, ambiguous tiles dropped
Prediction cache	(Prevented) stale predictions after relabel	Cache keyed by row order	Cache stores tile paths and re-predicts on mismatch
Excel / Jupyter	Updates lost	Open files saved over new versions	Close with "Don't Save"; write new files

18 What each indicator told us

Indicator	Good for	Misleading when
Accuracy (per label)	Viewpoint (balanced, one answer)	Rare classes: "always no" scores ~98%
Total validation loss	Early sanity	Dominated by the easy viewpoint head; hid trash-head progress
ROC-AUC (macro)	Ranking quality, independent of thresholds; comparing models	Says nothing about where to cut; can be high with useless F1 on rare classes
F1 (macro, per tile)	Actual yes/no quality	Very low for rare classes even with good models; not comparable across tile sizes
F1 per photo	Practical question ("does this photo have a bottle?")	Binary version: almost every photo has trash
`val_trash_auc`	Checkpoints, LR schedule, early stopping	Averages heads: one head can drift while the mean rises
Trash vs clean AUC/F1	"Is there debris?" level	F1 inflated where most tiles have trash (compare with "always yes")
Train vs validation gap	Overfitting (satellite) vs underfitting (shore)	Augmented training data makes train look slightly harder

19 Lessons and future work

Lessons

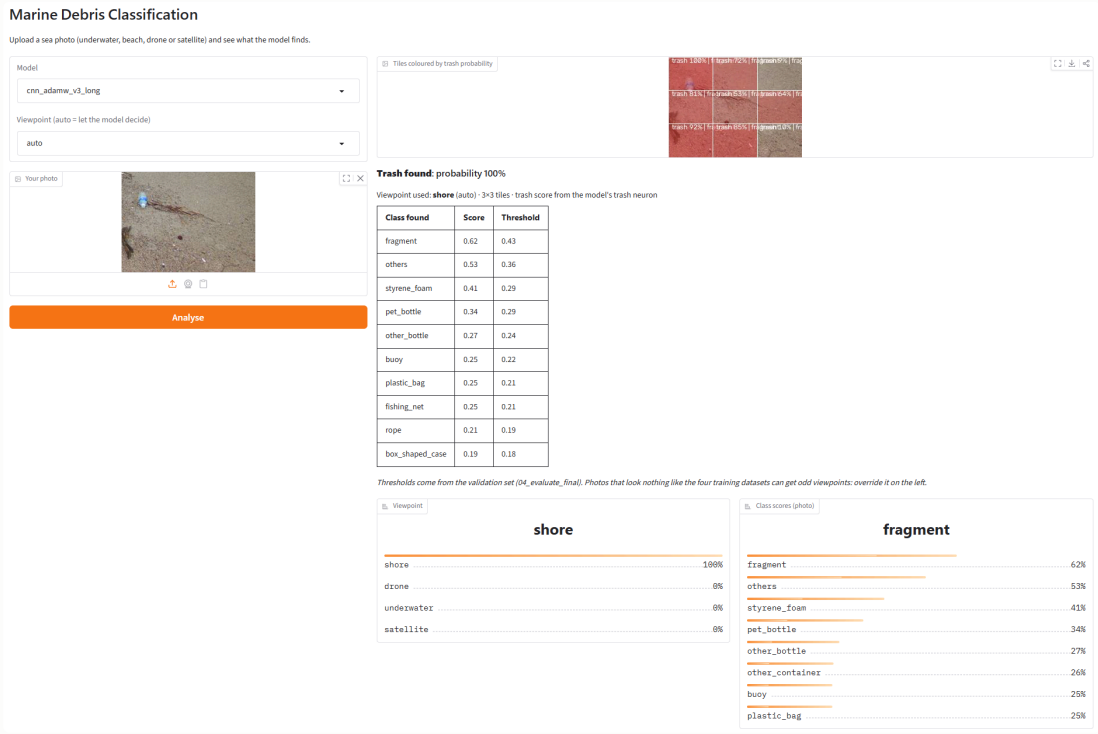
- **Look at the data before the architecture.** The biggest gains (dropping park images, 3×3 tiles, the relabel) were all data fixes found by looking at actual tiles.
- **Accuracy is not a metric for rare classes.** Report AUC and F1, and compare against the "always yes / always no" baseline.
- **Converting detection data to classification is a modelling decision** with its own bugs. Audit tile labels against the original annotations.
- **Leak-free splits lower the scores but make them real.**
- **Change one thing at a time** where possible (keeping drone's head unchanged in v3) so results can be attributed.
- **Fine-tune pretrained models gently**: frozen BatchNorm, small learning rate, and reset anything tied to the old features.

Future work

- **Train a detector** (YOLO-style) on the original boxes instead of converting to tile labels.
- **More diverse data**: more sites and dives, not more frames of the same videos.
- **Hard negatives** for underwater: explicit "animal / ROV" examples so "object ≠ trash".
- **Fewer, broader classes** where examples are scarce; calibrated or shared thresholds robust to class-mix shift.
- **Re-split SeaClear by site** so validation and test have similar class mixes.

20 Try it: the web app

A small local web app runs the trained models in the browser. Pick a model, upload any sea photo, and it shows the viewpoint, whether there's trash, the classes found (with the validation-tuned thresholds), and the photo cut into training-style tiles tinted by trash probability.



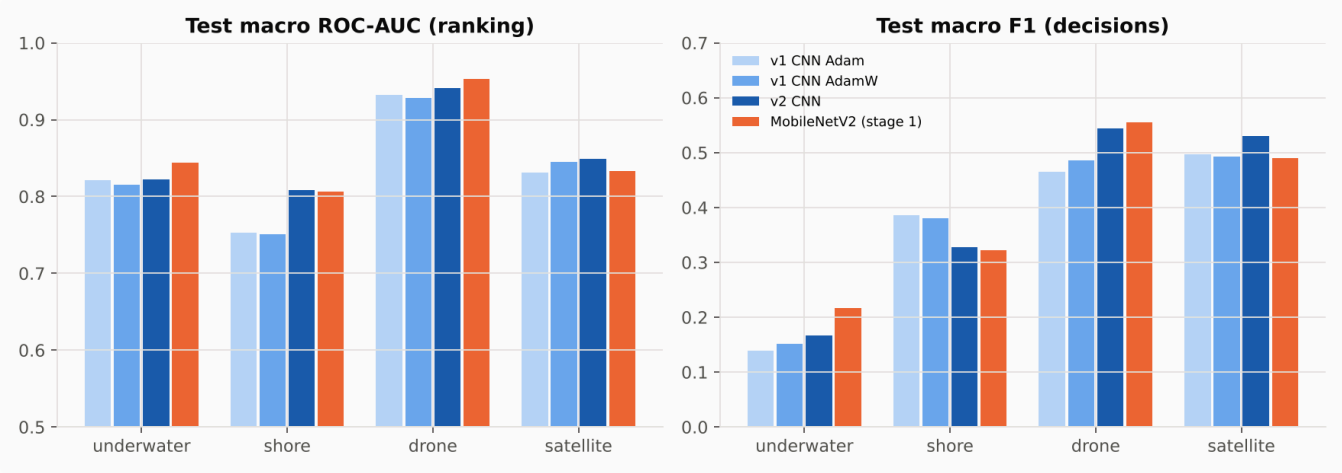
A real run with the final baseline CNN on a beach photo with a plastic bottle: viewpoint *shore* (100%), trash found, the 3×3 tiles around the bottle tinted red, and fragment, others, styrene foam and PET bottle above their thresholds.

Where everything lives now

The whole project lives in one folder, `Marine Debris Classification`. Every script finds the models and data relative to where it sits, so the folder can be moved or cloned without editing paths.

Folder	What is in it
`Datasets`	The four original datasets (named by viewpoint), `prepared` (tiles, manifests, classes.json), the data spec Excel sheet and the label-problem note
`Models`	The final models, `cnn_adamw_v3_long_best.keras` (baseline CNN) and `mnv2_adamw_v2_best.keras` (MobileNetV2), plus `cnn_adamw_v3_best.keras`, the 60-epoch CNN that `4_continue_cnn` continues from; `Evaluations` holds each one's test results and thresholds; the training logs and histories sit next to the models
`Notebook training codes`	In running order: `1_prepare_data.py`, `2_train_cnn`, `3_train_mobilenetv2`, `4_continue_cnn` and `5_evaluate`
`webapp`	`app.py` and its README
Top of the folder	This report (`Project_Journey_Report.pdf`), the presentation (`Marine Debris Classification.pptx`), the README and `requirements.txt`

A Appendix · Numbers



Test macro ROC-AUC and macro F1 for every model before v3, on the old labels. The v3 models are in section 14 and the second table below (corrected labels).

Model (old labels, test)	Underwater AUC / F1	Shore AUC / F1	Drone AUC / F1	Satellite AUC / F1	Viewpoint
v1 CNN Adam	0.821 / 0.138	0.753 / 0.386	0.932 / 0.465	0.832 / 0.497	99.6%
v1 CNN AdamW	0.816 / 0.152	0.751 / 0.380	0.928 / 0.486	0.845 / 0.493	99.6%
v2 CNN	0.823 / 0.166	0.808 / 0.328	0.941 / 0.544	0.849 / 0.531	99.6%
MobileNetV2 (stage 1)	0.844 / 0.217	0.806 / 0.323	0.953 / 0.555	0.833 / 0.489	98.5%

Model (corrected labels, test)	Underwater AUC / F1	Shore AUC / F1	Drone AUC / F1	Satellite AUC / F1	Viewpoint
v2 CNN, re-graded	0.821 / 0.190	0.808 / 0.328	0.941 / 0.544	0.849 / 0.531	99.6%
MobileNetV2 first run, re-graded	0.846 / 0.250	0.806 / 0.323	0.953 / 0.555	0.833 / 0.489	98.5%
CNN v3 (60 epochs)	0.841 / 0.207	0.798 / 0.309	0.946 / 0.566	0.864 / 0.560	99.4%
CNN v3 long (final baseline)	0.842 / 0.218	0.798 / 0.315	0.956 / 0.580	0.855 / 0.549	99.5%
MobileNetV2 v2 (final transfer)	0.847 / 0.271	0.801 / 0.319	0.958 / 0.600	0.800 / 0.452	99.6%

v3 data (SeaClear)	Train	Val	Test
Tiles (before → after relabel)	35,712 → 33,459	7,554 → 7,129	7,458 → 6,917
Clean share	74.0% → 60.9%	72.0% → 60.0%	72.1% → 62.1%
Ambiguous tiles dropped	2,253	425	541

G10 Marine Debris Classification · project journey report · generated from the project's logs, evaluation files and dataset annotations. Images: SeaClear, BePLi v2, UCWD and MADOS samples; screenshots taken during the project.